COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	10/1/2026	**DATE SUBMITTED:**	10/1/2026
NAME:	Al Twain D. Pasion	**INSTRUCTOR:**	Engr Jimlord Quejado

ACTIVITY NO. 10.1 Graphs

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

CODE

```
#ifndef GRAPH_H
#define GRAPH_H

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph{
// insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head

        return newNode;
    }

    int N; // number of nodes in the graph
    public:
        adjNode **head; //adjacency list as array of pointers

        // Constructor
        DiaGraph(graphEdge edges[], int n, int N) {

        // allocate new node
        head = new adjNode*[N]();
        this->N = N;

        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i)
        head[i] = nullptr;

        // construct directed graph by adding edges to it
            for (unsigned i = 0; i < n; i++) {
                int start_ver = edges[i].start_ver;
                int end_ver = edges[i].end_ver;
                int weight = edges[i].weight;

                // insert in the beginning
                adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);

                // point head pointer to new node
                head[start_ver] = newNode;
            }
        }
```

```
    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++){
            delete[] head[i];
            delete[] head;
        }

    }
};

// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
        << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}

#endif
```

```
#include <iostream>
#include "graph.h"

// graph implementation
int main() {

    // graph edges array.
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
    };

    int N = 6; // Number of vertices in the graph

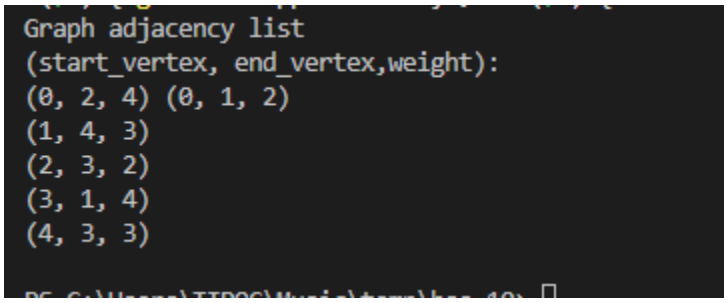
    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);

    // construct graph
    DiaGraph diagraph(edges, n, N);

    // print adjacency list representation of graph
    std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex, end_vertex,weight):'

    for (int i = 0; i < N; i++) {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }
    return 0;
}
```

OUTPUT



OBSERVATION

The errors in the code were just missing semi-colons and the destructor was not coded properly. The sequence of the code; first we define it in the structure “graphEdge” with (x, y, w), with it representing the starting vertex, ending vertex, and the cost/weight respectively. After that, we set up the number of vertices and number of edges so that we can pass it through the “DiaGraph” constructor. For every element, it turns the vertices into nodes and points them to their corresponding adjacent vertices. At the last part, it uses the display_AdjList for all of the newly created vertices with their corresponding weight.

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1

CODE

```
#ifndef dfS_H
#define dfS_H

#include <map>
#include <set>
#include <stack>
#include <string>
#include <vector>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e Edge<T> & : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}
```

```cpp
template <typename T>
class Graph
{
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N)
    {
    }

    // Return number of vertices in the graph
    auto vertices() const ->size_t
    {
        return V;
    }

    // Return all edges in the graph
    auto &edges() const ->const vector<Edge<T>> &
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const ->vector<Edge<T>>
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e :const Edge<T> & : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }
```

```cpp
    // Overloads the << operator so a graph be written directly to a stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest) ->vector<unsigned long long>
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push( x: 1); // Assume that DFS always starts from vertex ID 1

    while (!stack.empty())
    {
        auto current_vertex :size_t  = stack.top();
        stack.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e :Edge<T>  : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the stack.
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

```cpp
template <typename T>
auto create_reference_graph() ->Graph<T>
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i :pair<...> &  : edges)
    for (auto &j :pair<unsigned long long, T> &  : i.second)
    G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}

template <typename T>
void test_DFS()
{
    // Create an instance of and print the graph
    auto G :Graph<unsigned>  = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order :vector<size_t>  = depth_first_search(G,  dest: 1);
    for (auto v :size_t  : dfs_visit_order)
        std::cout << v << std::endl;
}
```

```cpp
#include <iostream>
#include "dfS.h"

int main() {

    using T = unsigned;
    test_DFS<T>();
    return 0;


}
```

OUTPUT

```
"D:\-COLLEGE\SECOND YEAR\CODING\HOA 10\cmake-build-debug\HOA_10.exe"
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

OBSERVATION

The DFS uses the concept of stack, so vertices are treated with “last-in, first out” order starting from the first vertex. For every popped vertex that has not been visited is marked as “visited”, and the unvisited adjacent elements are pushed onto the stack. In the main.cpp file, it printed the order 1,5,8,6,7,3,4, and 2. Since we started at 1, it was the first to be marked as “visited”. The reason why 5 was marked before 2 was because it was at the top of the stack, hence the “LIFO” behavior. This is then repeated until every element is marked as visited.

B.2

CODE

```cpp
#ifndef BFS_H
#define BFS_H

#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

// Edge of the graph
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};
```

```cpp
// Overload << so the graph can be printed
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (size_t i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e :Edge<T> & : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os; // FIX: missing in the activity sheet
}

// Graph using an edge list representation
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const ->size_t
    {
        return V;
    }

    auto &edges() const ->const vector<Edge<T>> &
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
```

```
    auto outgoing_edges(size_t v) const ->vector<Edge<T>>
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e :const Edge<T> & : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    // FIX: use U so it doesn't shadow the class template parameter T
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};
```

```
// Build the reference graph
template <typename T>
auto create_reference_graph() ->Graph<T>
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i :pair<...> & : edges)
        for (auto &j :pair<unsigned long long, T> & : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}
```

```cpp
// Breadth-first search starting from vertex 1
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest) ->vector<unsigned long long>
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push( x: 1); // Assume that BFS always starts from vertex ID 1

    while (!queue.empty())
    {
        auto current_vertex :size_t  = queue.front();
        queue.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

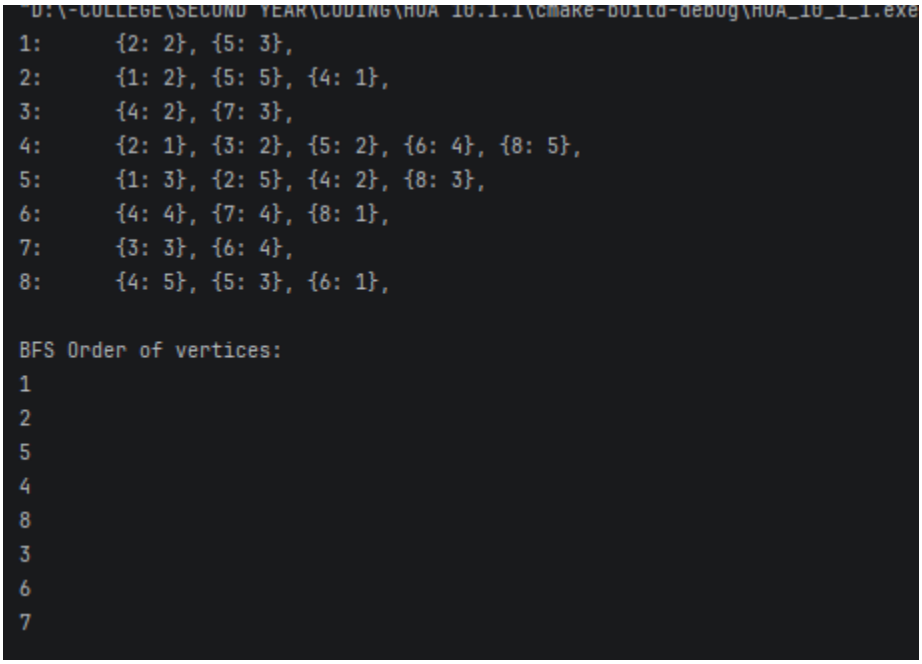
            for (auto e :Edge<T>  : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}

template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G :Graph<unsigned>  = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order :vector<size_t>  = breadth_first_search(G,  dest: 1);
    for (auto v :size_t  : bfs_visit_order)
        std::cout << v << std::endl;
}

#endif  #ifndef BFS_H
```

OUTPUT



OBSERVATION

The BFS searches through the elements using the same concept as the queue, "first-in, first out". Each time a vertex is dequeued, it is marked as "visited" and all of its adjacent vertices are enqueued. The error in this code is that in the operator<< function, it did not contain a "return os" and the parameter "template <typename T>" did not match our parameter "U", so by adding a return statement and changing the parameter, it compiled correctly. In the main.cpp file, it printed out 1,2,5,4,8,3,6, and 7. Our first element is 1, so we mark that as visited. Then we enqueue all the adjacent elements to 1 starting from the nearest to the farthest. Then we dequeue what's in front and mark that as visited, then we look for more unvisited vertices with this rule until everything has been searched.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

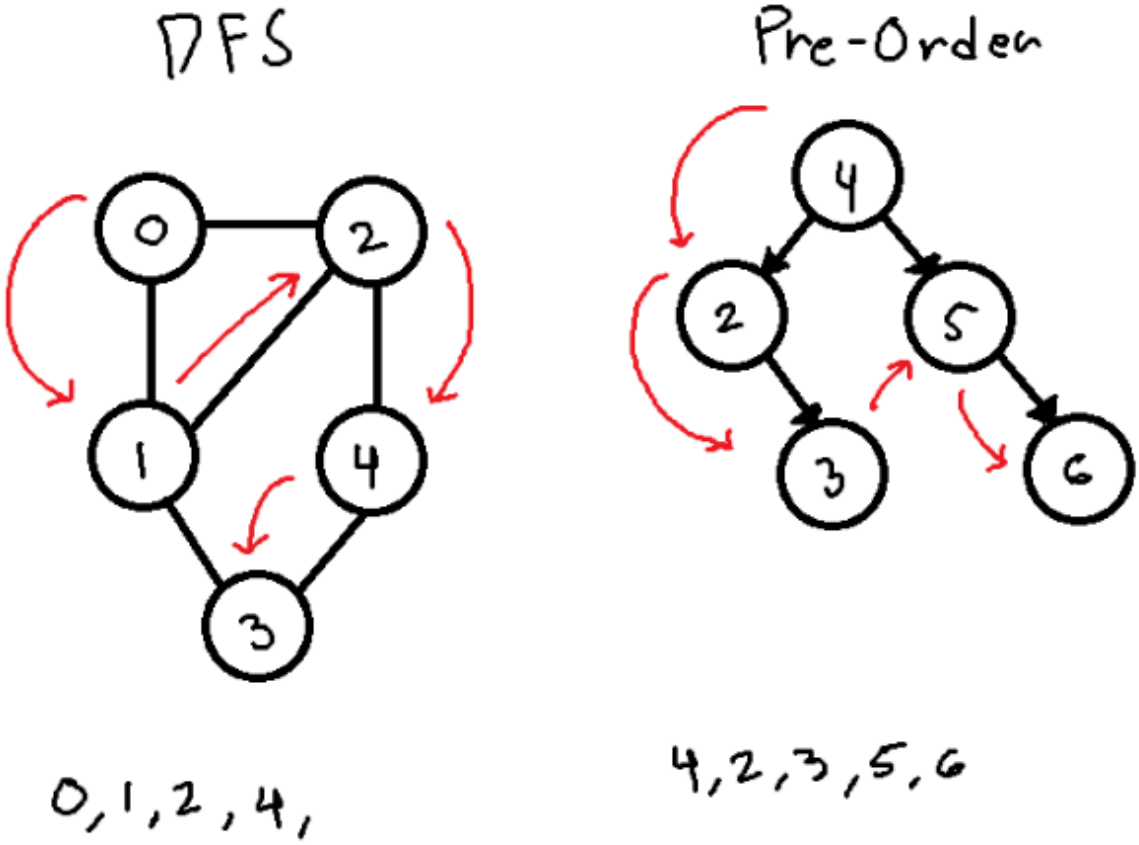
Copy and paste the questions from the lab and include your answers after.

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

The algorithm most helpful to accomplish this task is the Depth First Search. It starts at one location, and visits the adjacent vertices one by one. Then it back tracks to find more unvisited vertices until it has explored everything.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.



In the graphical representation I drew, we can see the flow of the depth first search and the pre-order tree traversal. They both start at a vertex/node, for the DFS it can be any vertex, while the pre-order starts at the root node. Then they find the lowest value that they are connected to and mark that as "visited", this then gets repeated starting from the last visited vertex/node until we've reached all vertices/nodes.

3. In the performed code, what data structure is used to implement the Breadth First Search?
A queue.

4. How many times can a node be visited in the BFS?
If we're talking about how many times we're gonna count it as visited, it should be one time.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.